

Synthesizing Staged Updates between Verified Controllers

ANONYMOUS AUTHOR(S)

Verifying an old controller and its replacement does not establish how to pass safely between them. Components may change at different times, while requirements overlap or begin with pending obligations. Fine-Grained Dynamic Update Controller Synthesis (FG-DUCS) constructs this transition from local transfers, requirement lifetimes and initial obligations. A winning policy completes safely from every reachable old state under every environment outcome and preserves the selected new controller’s continuation. We prove correspondence between the contract and its game, and a sufficient condition for serializing merged transfers. Established lazy exploration returns a ranked policy or a losing-region certificate. On 27 adapted games, it decides 26 within 1,200 s, versus 17 with eager exploration under the same pruning. Constructed examples separate granularities in both directions; 54 inherited-model comparisons show no decision change. The guarantee concerns finite event sequences in the model: commands and handover require uncontrollable quiescence, and continuous service or elapsed-time bounds are not promised.

CCS Concepts: • **Software and its engineering** → **Software evolution**.

Additional Key Words and Phrases: dynamic software updating, discrete-event controller synthesis, supervisory control, on-the-fly synthesis

ACM Reference Format:

Anonymous Author(s). 2027. Synthesizing Staged Updates between Verified Controllers. In *Proceedings of The ACM International Conference on the Foundations of Software Engineering (FSE 2027)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

Verified old and new controllers leave an important question open: how should control pass between them? During the transition, components may use different versions and requirements may have different lifetimes [3, 7, 15, 38]. In a workcell, an occupied station may need to pass its workpiece to another station before replacement. A new inspection rule may then begin with an inspection already owed. Safe endpoints do not settle these intermediate choices.

FG-DUCS generates a policy for this handover from local inputs. The modeler supplies the old and new controllers, permitted component transfers, requirement lifetimes and initial obligations. Synthesis chooses how to coordinate them from every reachable old state and for every environment outcome. At handover, it restores the selected new-controller state’s permitted continuation. This is stronger than merely satisfying its safety requirements: deleting a permitted action can preserve safety while changing the supplied controller (Section 3).

The intended use is checking a finite maintenance plan before execution. Update control takes over scheduling and enforces the active requirements; an interval requirement can protect a gap between old and new rules. Work may pause. Commands and handover occur only when the active-component product has no enabled uncontrollable (UC) event. Persistent environment activity can therefore prevent completion. The guarantee counts events, not elapsed time.

Dynamic Update Controller Synthesis (DUCS) [33, 34] and generalized-reactivity(1), or GR(1), updating [2] already generate update control. FG-DUCS fixes both endpoint controllers and constructs the intervening game from individual transfers and requirement boundaries. Section 2.3 makes this input/output distinction concrete. Our contribution is the contract, its guarantees and granularity conditions; the reachability algorithm is established.

Our contributions are:

C1: Contract and game. We prove local-history correspondence and endpoint preservation, and give an invariant-initialization criterion (Sections 3 and 4.1).

C2: Granularity conditions. We identify obstructions to merged commands and a local sufficient condition for serializing merged transfers (Sections 2 and 5).

C3: A policy with checkable evidence. Sound and complete lazy synthesis returns a completing policy and rank or a losing-region certificate (Section 6).

C4: Implementation evidence. Reference-game checks and 27 adapted contracts test the MTSA implementation; constructed comparisons separately test granularity (Section 7).

The replication package [4] contains Technical Appendix A–P and Supplement S1–S5, cited below; Data Availability gives versioned links.

2 Why Updates Need Separate Component and Requirement Choices

The following examples show why components and requirements need separate update choices. Section 2.3 then compares how FG-DUCS and existing update interfaces represent those choices.

2.1 Cell: Moving Work before Replacing a Station

Our constructed *Cell* example has two stations A, B and exactly one workpiece [23, 34]. A station can be replaced only while empty. The old controller moves the workpiece between stations and forbids inspection. The new controller requires inspection at B before departure. Both endpoints are verified, yet simultaneous replacement is impossible: one station always holds the workpiece.

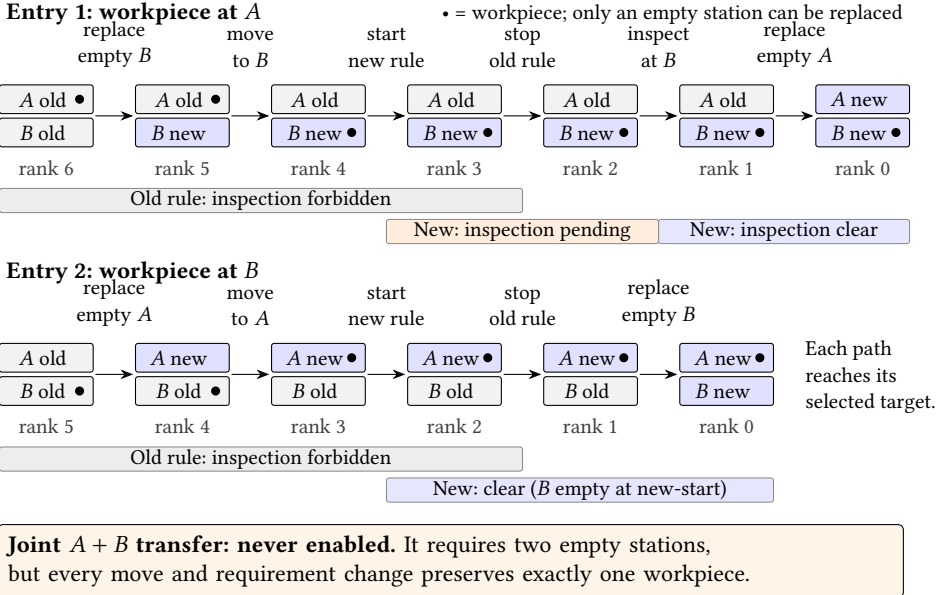


Fig. 1. A complete local policy from both old entries. Bands mark active rules and new-rule inspection memory; new-start precedes old-stop. Each arrow has one action and one outcome, decreasing rank toward its selected new target. The failed joint replacement uses the same endpoints and requirements.

Coordinating moves and changing duties. Figure 1 shows the policy from both old states, called *entries*. Each path replaces the empty station, moves the workpiece, and later replaces the other

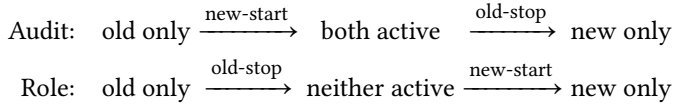
station. The input requires new-start before old-stop, preventing an unprotected gap; synthesis chooses the remaining order. If B holds at new-start, inspection becomes pending; it must wait until old-stop removes the ban. Starting its monitor—the automaton tracking the new rule—clear would silently discard this duty.

The first path therefore inspects before handover; the second activates the new rule with B empty and needs no inspection. Both reach an allowed new closed-loop state with inspection clear: z_A holds the workpiece at A , and z_B holds it at B . A nonnegative *rank* decreases on every arrow, excluding safe but endless inspection. Handover restores the selected endpoint’s full permitted behavior.

What synthesis chooses. The supplied ordering is partial, so synthesis still chooses the update sequence. All Cell events are controllable: m_A, m_B move to A, B , j inspects, and ρ_A, ρ_B replace the empty station. Technical Appendix B gives the complete input and path proof. The next example shows why requirements also need separate boundaries.

2.2 Audit and Role Lifetimes

We call the second constructed example *Policy(2)*: the number denotes its two old/new requirement pairs, audit and access role. At least one audit rule must remain active throughout, while the old and new roles must never overlap. These obligations require opposing boundary orders:



The role gap is declared by the interval requirement, rather than forced by access rules alone. The new audit monitor starts unlogged and requires a log before service. Each endpoint alternates its own log and service, permitting only its own access role. With an identity transfer, the saved policy obeys these opposing orders and reaches a compatible new endpoint from both old states in at most six events.

If the starts are merged and the stops are merged, the first start overlaps roles and the first stop removes audit coverage. Ordinary events and transfer do not change which rules are active, so neither old entry has a completing merged policy. Technical Appendix C gives the complete finite input, policy and losing closure. This compares merged and separate FG-DUCS requirement boundaries.

2.3 Existing Update Interfaces

Earlier endpoint reconnection. On-the-fly DUCS (O-DUCS) separates update and post-update analysis [18, Sec. 3]. The archived fork in S4.8 synthesizes a new controller, matches component and new-safety states, and copies its transitions [4]. Identity with the 2024 release is unestablished.

Independent requirement boundaries. DUCS’s transition requirement T constrains one global stop/start pair, including overlap, gaps and their duties [34, Def. 4.1]. FG-DUCS gives each requirement its own stop, start and initial obligation.

Cell through native interfaces. DUCS permits staged reconfiguration inside its supplied new environment E' [34, Sec. 4.2]. Technical Appendices H–I give chosen Cell encodings: our DUCS Cell encoding adds eventual handover beyond the safety- T implementation [34, Sec. 5.3]; GR(1) may restrict endpoint edges when synthesizing its new controller. In these encodings, the modeler writes the stages and boundaries into the supplied environment or predicates. FG-DUCS generates them

Table 1. Supplied inputs and generated results: DUCS [34, Defs. 4.1–4.3, Sec. 5], GR(1) updating [2, Def. 3.1, Sec. 4], and FG-DUCS.

Interface	Supplied inputs	Generated structure and successful result	Progress and continuation
DUCS	Old controller; old/new environments and goals; global transfer relation; transition requirements	Update environment and goal; controller and entry map covering every old-controller state	After the update trigger: eventually stop, start and reconfigure; enforce new goals from new-start.
GR(1) updating	Old/new GR(1) specifications and switching condition	New controller, bridge and switching bookkeeping	The generated new controller runs unchanged after the bridge.
FG-DUCS	Fixed components/controllers; component transfers; monitors, initializers and precedence; allowed handover states	Mixed-version and boundary game; policy, rank and handover map for every old entry and permitted outcome	Supplied new closed loop is preserved from the selected compatible target.

from local inputs. These controllable, deterministic witnesses establish neither a general translation nor a cost advantage (S5).

3 Defining an Update Contract

3.1 Supplied Choices

The contract states what may change and what must be preserved. Its inputs are component models and transfers, requirements and their initializers, and fixed endpoint controllers. Synthesis supplies the coordination among them.

Finite LTSs and safety monitors. A labeled transition system (LTS) $\mathcal{L} = (S, \Sigma_{\mathcal{L}}, \delta, s^0)$ has nondeterministic transitions $\delta : S \times \Sigma_{\mathcal{L}} \rightarrow 2^S$. Partition events as $\Sigma = \Sigma_{uc} \uplus \Sigma_c$: controllers may disable controllables but cannot select outcomes or suppress uncontrollable (UC) events. A synchronous-product event requires a transition in every component whose alphabet contains it; other components retain their state [25]. A monitor is an automaton that checks a requirement as events occur. Requirement r has a total deterministic monitor $t_r = (M_r, \Sigma, \eta_r, m_r^0, F_r)$ with absorbing errors for violating prefixes [1, 22]. The empty monitor product has one non-error state. Monitors observe labels; policies also observe successor component states.

Components and transfers. For each $i \in I$, let $e_i^{\text{old}}, e_i^{\text{new}}$ be the pre-update and post-update components and let $g_i \subseteq S_{e_i^{\text{old}}} \times S_{e_i^{\text{new}}}$ be their state-transfer relation. Define $g_i(s) = \{s' \mid (s, s') \in g_i\}$. The supplied g_i includes every modeled transformation outcome. Let $E^v = \parallel_{i \in I} e_i^v$.

Requirements and activation. Let pairwise-disjoint $R^{\text{old}}, R^{\text{new}}, R^{\text{upd}}$ contain the pre-update, post-update, and update-interval safety requirements, respectively. For each $r \in R^{\text{new}} \cup R^{\text{upd}}$, the input supplies a partial *monitor-initialization map* ι_r from version-tagged component tuples to $M_r \setminus F_r$. Its domain specifies where activation is allowed. Section 4.1 defines history-based and activation-scoped interpretations. For $r \in R^{\text{upd}}$, $\text{dom } \iota_r$ includes every reachable old component state.

Endpoints and handover targets. For $v \in \{\text{old}, \text{new}\}$, the supplied finite controller C^v preserves uncontrollables and makes $E^v \parallel C^v$ deadlock-free and satisfy R^v . Hiding controller auxiliary events leaves observation alphabet $\bigcup_i \Sigma_{e_i^v}$; the union over both versions is the ordinary alphabet Σ_N . Write Reach for initial-state reachability and

$$\text{CL}^v = E^v \parallel C^v \parallel \parallel_{r \in R^v} t_r.$$

Monitors are passive: every endpoint transition originates in $E^v \parallel C^v$; a monitor only updates on that label and adds no enabled event. The input supplies nonempty, checked targets $Z_{\text{load}} \subseteq \text{Reach}(\text{CL}^{\text{new}})$ with component, controller and new-monitor states.

Update commands and order. The set of update events, each executed exactly once, is

$$\Sigma_{\text{upd}} = \{\rho_i \mid i \in I\} \cup \{\text{stop}(r) \mid r \in R^{\text{old}}\} \cup \{\text{start}(r) \mid r \in R^{\text{new}}\}. \quad (1)$$

These controllable events transfer components, stop old monitors or start new ones, respectively, and are disjoint from ordinary events Σ_N : $\Sigma = \Sigma_N \uplus \Sigma_{\text{upd}}$ and $\Sigma_{\text{upd}} \subseteq \Sigma_c$. Each transfer event ρ_i is one atomic model transition: the controller chooses the update, but cannot choose its g_i outcome.

An optional strict partial order $< \subseteq \Sigma_{\text{upd}} \times \Sigma_{\text{upd}}$ expresses precedence among update operations. For a pending-event set P , event a is order-enabled exactly when $a \in P$ and every $a' < a$ has left P . Monitors self-loop on update events that they do not specify.

Eligible contracts. An eligible contract satisfies the finiteness, typing, totality and non-error conditions above; dependencies form a strict partial order, and endpoint controller alphabets cover environment labels. Eligibility is structural; Section 4.1 justifies initializer meanings.

3.2 States, Transitions, and Completion

States and all entry states. An update state $q = (\xi, \mu, P)$ records each component's version/state $\xi(i) = (v, s)$ with $s \in S_{e_i^v}$, each monitor's state $\mu(t_r) \in M_r \cup \{\perp\}$ ($\perp$ means inactive), and pending commands $P \subseteq \Sigma_{\text{upd}}$. Here $\rho_i \in P$ exactly when component i is old. Old monitors remain active until their stop leaves P ; new monitors become active when their start leaves P ; update monitors are always active.

Admission $\text{Init}(z)$ copies z 's component and old-monitor states, installs interval initializers, leaves new monitors inactive and sets $P = \Sigma_{\text{upd}}$. The update policy replaces C^{old} and its scheduling restrictions. Declared old monitors remain until their stops; undeclared objectives, including old liveness, are not preserved. Duties needed throughout belong in interval monitors. The entry set is

$$Q_0 = \{\text{Init}(z) \mid z \in \text{Reach}(\text{CL}^{\text{old}})\}. \quad (2)$$

All-entry coverage forbids waiting for a favorable old state.

Four kinds of induced transitions. For $\xi(i) = (v, s)$, component i uses e_i^v . The active-component product determines enabled ordinary uncontrollables $U(q)$, including those with unsafe outcomes, independently of updates and goals. For every $a \in \Sigma_{\text{upd}}$, impose

$$U(q) \neq \emptyset \implies \text{Post}(q, a) = \emptyset. \quad (3)$$

Thus even a local command waits until the active-component product has no enabled uncontrollable event. A finite operation may drain to such a point; perpetual polling may prevent it. This global *uncontrollable quiescence* differs from transaction quiescence [21] and tranquility [42]. Ordinary controls may coexist with UC. The following rules define Post ; unmentioned coordinates stay unchanged. “Advance” means $\mu'(t_r) = \eta_r(\mu(t_r), a)$ for the specified active monitors.

Ordinary event $a \in \Sigma_N$. If a belongs to the local event set of at least one active component and every active component having a can transition, take every synchronous-product outcome and advance every active monitor.

Component update ρ_i . If $U(q) = \emptyset$, order-enabled, and $\xi(i) = (\text{old}, s)$, set $\xi'(i) = (\text{new}, s')$ for each $s' \in g_i(s)$, delete ρ_i from P , and advance every active monitor.

Requirement deactivation $\text{stop}(r)$. If $U(q) = \emptyset$ and order-enabled, advance every active monitor except t_r , retain inactive monitors at $\perp$, set $\mu'(t_r) = \perp$, and delete $\text{stop}(r)$ from P ; t_r does not observe it.

Requirement activation $\text{start}(r)$. If $U(q) = \emptyset$, order-enabled, and $\xi \in \text{dom } t_r$, advance every active monitor, retain other inactive monitors at $\perp$, install $\mu'(t_r) = t_r(\xi)$, and delete $\text{start}(r)$ from P ; t_r begins online observation afterward.

If a precondition is false, the successor set is empty; every nondeterministic outcome is included and environment-chosen. Let Q contain every state reachable from Q_0 through these transitions. *Safety and handover-ready goals.* An update state $q = (\xi, \mu, P)$ is safe, written $\text{Safe}(q)$, exactly when every active monitor is outside its error states. For $P = \emptyset$ and $z \in Z_{\text{load}}$, write $q \sim z$ when component states (without version tags) and all new-monitor states agree. The handover map κ selects such a full endpoint state z . Goals are

$$Q_{\text{goal}} = \{q = (\xi, \mu, \emptyset) \in Q \mid \text{Safe}(q) \wedge U(q) = \emptyset \wedge \exists z \in Z_{\text{load}} : q \sim z\}. \quad (4)$$

The goal is a quiescent handover boundary, not merely completion of the command list. At a goal, interval obligations end and matched new monitors continue. Handover disables ordinary controllables and installs $\kappa(q)$'s controller state, outside Σ and the rank. UC cannot compete; compatible targets may have different continuations.

State-dependent policies and finite completion. A memoryless update policy is a function $\sigma : Q \setminus Q_{\text{goal}} \rightarrow 2^{\Sigma_c}$ selecting only events with nonempty successor sets.

The possible next states under this policy are

$$\text{Succ}_{\sigma}(q) = \bigcup_{u \in U(q)} \text{Post}(q, u) \cup \bigcup_{a \in \sigma(q)} \text{Post}(q, a). \quad (5)$$

The environment chooses any uncontrollable and any permitted-event outcome. The returned policy retains one controllable at a quiescent state.

Let $\text{Reach}_{\sigma}(Q_0)$ be the policy closure from Q_0 , stopping at goals.

Runs, winning, and witnesses. A run follows the policy successors and stops at a goal. It is *maximal* if infinite or ending at a goal or a state without successors. A single policy σ is winning for an FG-DUCS instance if, from every $q_0 \in Q_0$, (i) every reachable update state is safe, (ii) every reachable non-goal state has a successor, and (iii) every maximal update run reaches Q_{goal} after finitely many events. This *finite completion* counts events and assumes no fairness. Winning policies exclude controllable self-loops during update; a reached UC self-loop makes a state losing. Ordinary actions can progress, as Cell's moves and inspection do. Handover restores the supplied controller's full matched behavior.

A *progress-ranking function* for a winning policy is $\text{rk} : \text{Reach}_{\sigma}(Q_0) \rightarrow \mathbb{N}$, with value 0 at goal states and satisfying $\text{rk}(q') < \text{rk}(q)$ for every non-goal state q and every $q' \in \text{Succ}_{\sigma}(q)$.

The FG-DUCS problem. Decide whether one policy safely completes the eligible game from every entry under every permitted event and outcome. Return $(\sigma, \text{rk}, \kappa)$ covering all Q_0 , or a losing-region certificate.

3.3 Why the Generated State Represents the Contract

A *local update history* $\ell = z(a_1, o_1) \cdots (a_n, o_n)$ records the old entry, events and realized local successors. Components follow their active-version transitions and transfers; monitors fold their declared active intervals. Commands obey precedence, guards and UC quiescence. A source policy selects controllable labels, retains every UC and permitted outcome, and safely completes every maximal history at a matching quiescent target. Technical Appendix A defines these contract operations on histories.

PROPOSITION 3.1 (CONTRACT/GAME CORRESPONDENCE). *For an eligible contract, map each local update history to its current component tuple, active monitor memories and unexecuted commands. Histories with the same image admit the same next events and all the same local outcomes, agree on safety, and have the same handover targets. This map gives a label- and outcome-preserving correspondence between local update histories and paths of the generated game. A history-dependent policy satisfying the source obligations exists exactly when a winning policy exists in that game.*

PROOF. Fold a source history into its current tuple $F(h)$. At entry this is $\text{Init}(z)$. Each ordinary step uses the same participating component edges and full Cartesian product of outcomes; each transfer uses the same quiescence/order tests and g_i , advances active monitors and consumes its command. Stop removes its own monitor before observation; start installs its initializer after observation by existing monitors. Thus every label and outcome has exactly the same successor image, and safety and matching targets agree.

Induction maps source histories to game paths. Conversely, choose an old tuple witnessing the entry and lift each edge by this one-step correspondence, also for infinite paths. A game policy lifts through F ; applying a source policy to consistently lifted game histories gives a history-dependent game policy. Both directions preserve UC events, all selected-label outcomes, unsafe prefixes, deadlocks and infinite avoiding runs. Finite reachability attractor layers yield a state-based winner exactly when any history-dependent winner exists (the independent memoryless-policy lemma in Technical Appendix A), completing the equivalence. $\square$

This proves correspondence for the supplied contract; requirement meaning and Java frontend correctness remain separate obligations.

PROPOSITION 3.2 (PRESERVATION OF THE SELECTED ENDPOINT). *For a returned WIN certificate and reached goal q , the new-phase submodel reachable after handover at q is label-preserving isomorphic to CL^{new} rooted at $\kappa(q)$. Hence their finite and infinite ordinary-event traces coincide.*

PROOF. Handover retains matching components and monitors and installs the selected controller state. Each new-phase state copies an endpoint state with exactly its labeled transitions, without further policy restriction or return to updating. This bijection preserves and reflects edges, hence finite and infinite paths. $\square$

Continuation properties established at the selected state are preserved; target selection can change which continuation applies.

Why matching and reconnection both matter. Cell's prefix $\rho_B, m_B, \text{start}, \text{stop}, \rho_A$ finishes every command with inspection pending. Physical matching alone admits clear target z_B , whose m_A violates that duty; monitor matching excludes the reset. Conversely, disabling controllable loop b in a one-state endpoint with loops a, b and goal *true* preserves legality, deadlock freedom and safety [34, Defs. 3.9–3.10, 4.3], but loses b -continuations. FG-DUCS's fixed-endpoint contract requires preserving those continuations.

Technical Appendix G characterizes policy reuse under changed endpoints, including a saved production-cell (PC2) instance.

4 What a Winning Policy Guarantees

A safe game state is useful only if its monitors represent the intended duties. We first explain how to initialize those duties, then state the assumptions connecting a model policy to execution. Table 2 summarizes what each layer guarantees.

Table 2. Guarantee scopes and their premises. Table 4 maps RQ2 checks to these scopes.

Scope	What follows	Additional premise or boundary
Game	Safe finite completion for all entries/outcomes; matched handover	The supplied transition and endpoint rules (Theorem 6.1)
Invariant from start	Truth at activation and while active	Correct predicates/observations and initializer conformance
Safe prior histories	Continuations preserve obligations of the admitted safe histories	Residual inclusion; an empty history scope makes inclusion vacuous
From update entry	Interval monitors reflect obligations from admission onward	Correct boundary state and entry interpretation; S3 gives sufficient conditions
Execution	Handover within the entry’s event rank; selected endpoint continues	A1–A4; no elapsed-time bound or guarantee of platform conformance

4.1 Giving Requirements Their Intended Lifetimes

An initializer records what is owed when a requirement starts. Cell makes this concrete: a workpiece already at the inspection station brings an inspection duty with it.

A duty installed at activation. Cell’s owed-inspection bit starts at 1 exactly when B holds the workpiece. Event m_B sets it, j clears it, and m_A is forbidden while it is 1; other labels preserve it. Encoding 0 by c and 1 by d makes every monitor transition identical to this rule, including rejection of forbidden departure. Initialization and induction over subsequent events show that the monitor enforces exactly this duty rule. Figure 1’s first path starts with debt and clears it by ordinary inspection; the second starts clear.

Invariants that begin at activation. An invariant that begins at activation must hold immediately after start and after every later event. We call this the activation-scoped interpretation, written ACT. For a pure invariant $\Box\varphi$, its truth depends on the current values of event-tracked properties (fluents). Deterministic observers update these values through $d : V \times \Sigma \rightarrow V$. From the actual post-start vector v , the permitted continuations are

$$\text{ACT}_\varphi(v) = \{w \mid \forall u \preceq w : \varphi(d^*(v, u))\},$$

where prefixes include the empty word. A faithful safety monitor M has non-error language $S_\varphi = \text{ACT}_\varphi(v_0)$ from its initial state. Let $L_M(m)$ denote its continuations from state m .

PROPOSITION 4.1 (SAFE LOOKUP CELLS FOR PURE INVARIANTS). *If a lookup at v selects a non-error state m reached together with v by a safe word in the unrestricted monitor–observer product, or a state with the same continuation language, then $\varphi(v)$ holds and $L_M(m) = \text{ACT}_\varphi(v)$.*

PROOF. Let h reach the pair (m_h, v) . Faithfulness and non-error reachability give $h \in S_\varphi$, hence $\varphi(v)$. For every suffix w ,

$$w \in L_M(m_h) \iff hw \in S_\varphi \iff w \in \text{ACT}_\varphi(v).$$

The second equivalence uses safety of the prefix h and deterministic observers: future valuations depend only on v and w . Language equivalence transfers the equality to m . $\square$

This criterion checks the selected lookup state against a reference observation model. Its witness need not be plant-executable or obey one-shot update restrictions. The argument uses residual-language reasoning [13, 37], applied to the generated lookup. At activation, the adapter must choose the actual post-start key and use the same future observation rules; start is consumed once. Undefined entries and fallback initialization are outside the criterion. A true $\varphi(v)$ alone does not supply a lookup entry.

RQ2 distinguishes finite source checks from conformance at actual activations (Technical Appendix E.1).

When prior histories are part of the requirement. Some duties depend on what happened before activation. The initialized monitor must then allow only continuations consistent with every admitted past. Technical Appendix B.1 formalizes this as residual inclusion: the initialized monitor’s continuations must satisfy the requirement after every history in its declared scope. Equality means exactness for that declared history scope; stricter inclusion can reject feasible updates. This is a different reference from Cell’s duty rule or an invariant that begins at activation. The scope must contain the actual histories, and an empty scope makes the inclusion vacuous.

The saved checks use safe pre-activation histories consistent with the current observation for new requirements and a reference beginning at entry for interval requirements. A whole-history interpretation also includes prior violations and is considered separately (S3). These interpretations answer different questions. In Cell, activation installs a pending inspection duty exactly when B holds the workpiece; inspection then clears that duty. A rule that starts without prior debt would instead initialize clear. The modeler chooses the intended duty before synthesis. The checks above establish whether the monitor represents that chosen duty.

4.2 Execution and Handover

To use a model policy, an implementation must preserve what the policy observes and controls. The execution connection has four premises (Technical Appendix F):

A1: Observations. Endpoint observations preserve component and monitor states and same-labeled endpoint edges.

A2: Event progress. Update control disables only unselected controllables, retains every UC event and outcome, and eventually observes a next event and its successor.

A3: Commands. Commands start quiescently and finish atomically, finitely and without failure, reporting a modeled successor.

A4: Boundaries. Entry copies the actual old state; quiescent handover installs the selected compatible state. Both boundaries are finite, failure-free and linearizable [17].

Under A1–A4, execution follows the linked model and reaches handover within the entry rank (Theorem F.1). Event progress in A2 gives no fairness preference among choices and no time bound. For a history-based duty, the declared scope must also contain every actual activation history. Internal divergence and unobservable outcomes fall outside these premises.

5 How Granularity Changes Feasibility

Merging removes intermediate states and can prevent or enable completion.

What is merged. Partition transfers, old stops and new starts separately into blocks. A transfer block $B \subseteq I$ simultaneously permits all targets in $\prod_{i \in B} g_i(s_i)$; a boundary block removes or initializes its monitors at the same component tuple. Other active monitors compose member-label

effects, checked to commute pairwise on every monitor/residual state. Mapped precedence must remain irreflexive and acyclic. Ordinary behavior, entries, endpoints, relations, initializers and targets stay fixed; quiescence and all member guards apply. These are comparison contracts, not DUCS encodings.

LEMMA 5.1 (MANDATORY-COMMAND OBSTRUCTION). *Let q_0 be an entry and B a nonempty set of commands pending there. Suppose a safe region R contains q_0 , retains all commands in B , and is closed under every safe successor whose label is outside B . If each command in B is disabled or has an unsafe outcome at every state in R , no winning policy covers q_0 .*

PROOF. Every goal exhausts pending commands, so a winner must execute a first command in B . Safety and closure keep its prefix in R , where that step is disabled or has an environment-chosen unsafe outcome. Avoiding it yields a non-goal deadlock or infinite run, also losing. $\square$

In Cell’s one-workpiece region, joint transfer is disabled. Policy retains its initial active bits until either grouped boundary violates the interval. PC2-Rolling’s enabled joint transfer violates availability (Section 7.2.1). These results concern specified merges, not native DUCS expressiveness.

A capacity boundary. Readiness gates motivate the constructed Rolling contract [32, 39]. It starts with n ready replicas, no spares, and an interval floor of m ready replicas. A transfer boots its replicas; each becomes ready after one UC event, with no waiting loop. Service self-loops are controllable. The all-new, ready target restores capacity; service may pause during the update.

PROPOSITION 5.2 (ROLLING THRESHOLD). *For $n \geq 2$, $1 \leq m < n$ and $1 \leq k \leq n$, fixed consecutive batches of size k (with a smaller final remainder) admit safe finite completion iff $k \leq n - m$.*

PROOF. Some mandatory batch has size k . Its transfer leaves at most $n - k$ ready replicas, so $k > n - m$ violates the interval. Otherwise, transfer one batch only when all replicas are ready, disable service, and retain every UC readiness outcome before the next transfer. Readiness never falls below $n - k \geq m$. With u old and b booting replicas, $2u + b$ strictly decreases on every selected transfer and every UC event, reaching zero exactly at the target. Thus all maximal runs finish without fairness. $\square$

When separate transfers preserve feasibility. Finer commands need not preserve a merged winner. Two one-state components can synchronize UC labels so that both endpoints are quiescent, but either mixed version has a UC loop. Separate identity transfers then lose; a joint transfer skips the mixed state and wins at rank one (complete contract in Technical Appendix O.12).

PROPOSITION 5.3 (SERIAL TRANSFER PRESERVATION). *Consider transfer-only merging with unchanged entries, endpoints, targets, initializers and ordinary behavior, Cartesian transfer outcomes, commuting active-monitor effects and valid mapped precedence. If every transfer preserves its component’s UC synchronization alphabet and enables no previously disabled local UC label, a merged winner implies a fine winner preserving the same selected endpoint continuations.*

PROOF. The local condition keeps partial batches UC-quiescent. Any partial monitor error would persist to a completed Cartesian outcome, contradicting the merged winner. Finite serial expansion retains completion and endpoint choices; Technical Appendix O.13 gives the full argument. $\square$

The constructed Threads family, with workers sharing a bounded queue, satisfies this condition: old/new worker graphs coincide and transfers preserve idle states and the queue (Technical Appendix O.11). Rolling enables readiness and violates it, but its capacity threshold separately gives safe batch sizes.

6 Synthesizing and Checking an Update Policy

The procedure specializes strong-planning and on-the-fly reachability principles [8, 9, 41], retaining every queried event outcome and producing evidence for every old entry.

A *bucket* is $\text{Post}(q, a)$; empty means disabled. Disabling controls at a UC-enabled state removes runs without creating deadlock, preserving winning (*UC priority*, Lemma A.1). With quiescent goals and no fairness or controller priority, permitting commands while UC is enabled therefore adds no winning entries. Changing fairness or priority changes the game. The proof establishes memoryless sufficiency.

6.1 Lazy Successor Generation

Algorithm 1 constructs a reachability attractor [6]. V contains discovered states, X those with all UC candidates queried, and W proved winners. *Promote* adds an unproved safe state in X to W when its nonempty UC successor set lies in W or, at quiescence, one complete nonempty control bucket lies in W . It retains that proving set and assigns rank one above its maximum successor rank. Predecessors are then reconsidered. Lazy queries controls incrementally, keeping unfinished cursors in *Retry*. Failure requires exhausted candidates and no further promotion. Extraction retains lower-rank controls and matching endpoint targets.

Algorithm 1: Lazy

Input: Typed update contract (Section 3)

Output: $(\sigma, \text{rk}, \kappa)$ or (no solution, L)

```

1  $V \leftarrow Q_0$ ;  $X, \text{Retry} \leftarrow \emptyset$ ;  $W \leftarrow V \cap Q_{\text{goal}}$ ;
2  $\text{Open} \leftarrow \{q \in Q_0 \mid \text{Safe}(q) \wedge q \notin Q_{\text{goal}}\}$ ; set  $\text{rk}(q) \leftarrow 0$  for  $q \in W$ ;
3 while true do
4   Add queued goals to  $W$  once with rank zero; exhaust Promote, retaining witnesses/ranks, removing winners
   from Retry and propagating predecessors;
5   if  $Q_0 \subseteq W$  then return  $(\sigma, \text{rk}|_{\text{Reach}_\sigma(Q_0)}, \kappa)$  constructed from the goal witnesses and retained lower-rank
   choices;
6   if  $\text{Open} \neq \emptyset$  then
7     Remove  $q$  from Open, add it to  $X$ , and query all uncontrollable candidates;
8     Close the control cursor if UC-enabled; otherwise initialize it and query to the next nonempty bucket or
     exhaustion;
9     Insert  $q$  into Retry iff  $q \notin W$  and its cursor has remaining candidates;
10  else
11    if  $\text{Retry} = \emptyset$  then return (no solution,  $L \leftarrow V \setminus W$ );
12    Remove  $q$  from Retry; query to the next nonempty bucket or exhaustion; reinsert iff  $q \notin W$  and its
    cursor has remaining candidates;
13  Add every queried successor to  $V$ ; queue newly discovered goals for rank zero and safe non-goals for Open
    (unsafe states stay outside  $W$  and Open);
```

6.2 Correctness and Certificates

THEOREM 6.1 (TERMINATION, SOUNDNESS, AND COMPLETENESS). *For every eligible finite FG-DUCS instance with Equation (3) and quiescent goals in Equation (4), Algorithm 1 terminates. A returned $(\sigma, \text{rk}, \kappa)$ keeps every active monitor non-error, preserves every uncontrollable event and every outcome of each policy event, reaches Q_{goal} finitely without deadlock, and gives each reached goal a compatible target in Z_{load} for the supplied game. It returns no solution iff no winning policy covers Q_0 in that supplied game.*

PROOF. *Coverage.* Every new safe non-goal enters Open and leaves only after expansion queries all UC candidates. At quiescence, finite control cursors retain unfinished nonwinners in Retry; promotion removes only winners. Thus expanded states have complete UC outcomes, unfinished nonwinners remain retryable, and unexpanded safe non-goals remain open.

Termination. Each state is expanded/promoted once and each candidate queried once. Reverse incidences are processed on promotion, so the finite search terminates.

Soundness. Goals start at rank zero with compatible handover states. By induction, each promotion adds a safe state above a complete nonempty proving set of previously ranked winners: every UC successor, or every outcome of one control event. Queried buckets are immutable. Extraction retains all UC events, chooses one such control bucket at quiescence, and disables other controls. Every reached non-goal is safe, has a successor, and every successor has strictly smaller natural-number rank. Consequently no maximal run is an infinite run or a non-goal deadlock, and every run reaches a matching quiescent goal. The monitor rules advance exactly the memories active over each event; induction from entry establishes every declared safety interval. Handover retains the matching new memories.

Completeness. At failure both exploration queues are empty and promotion has reached a fixed point. Some entry is in $L = V \setminus W$, and L contains no goal. Coverage ensures that each safe state in L was expanded. If it is UC-enabled, at least one UC outcome lies in L , since otherwise it would be promoted. At quiescence every control candidate is exhausted; each enabled control bucket has an outcome in L for the same reason. Against any history-dependent policy the environment can select such an outcome after every history. Disabling all events deadlocks; otherwise the resulting run reaches an unsafe state or stays outside the goals forever. Hence no winning policy covers that entry. A winning policy therefore rules out failure and, by termination, forces success. Finally, UC priority loses no winning policy: removing optional controls at a UC-enabled state removes runs without introducing a deadlock, and the restricted policy remains admissible in the original game. $\square$

Checking the returned evidence. We write WIN for a completing policy and LOSS for proved nonexistence of one. WIN checking requires exactly $D_\sigma = \text{Reach}_\sigma(Q_0)$ as the rank domain, safety, enabled selected controls, nonempty all-outcome closure and strict decrease at non-goals. Goals require rank zero, quiescence and a compatible κ , defined exactly on $D_\sigma \cap Q_{\text{goal}}$; omitted policy entries are empty. LOSS checking requires an entry-containing, goal-free L with the UC/all-control counter-outcomes used in the completeness proof. These certify search relative to the successor and endpoint predicates [19]; Section 7 states their implementation trust boundary.

Exploration cost. Lazy may still explore the whole reachable graph. Technical Appendices K and O give the complexity analysis and an exponential separation example.

7 Implementation and Evaluation

7.1 Design and Implementation

We examine feasibility (RQ1), the scope of implementation checks (RQ2), and exploration cost (RQ3). Constructed examples isolate mechanisms; 27 adapted contracts test performance.

Each inherited input has three configurations. Base adds no interval requirement. R1 protects selected actions during gaps between old and new requirements. R2 requires selected requirement boundaries before component transfers. These are supplied requirements, not generated rules; Technical Appendix L gives their model-specific events and input sizes. A production-cell case separately tests availability. The archived O-DUCS implementation (Legacy) solves a different subsequent-control problem; its results are separate references.

Implementation. Our MTSA extension [11] checks certificates [29]; its composition checker, Link, checks the connected old, update and new controllers, including entries and targets. An independent oracle parses a restricted subset of Finite State Processes (FSP) [25]. Finite-JSON runs save losing membership; FSP benchmarks report only its statistics.

Use of generative AI. Generative AI assisted contract and example design, proof drafting, Java solver/frontend implementation, Python checkers, experiment drivers, execution, result analysis and manuscript editing. Validation used certificate and exhaustive checks for algorithms, source and automaton comparisons for initialization, saved-output checks for tables and primary sources for citations. Technical Appendix P details tool roles and validation; Threats discusses correlated faults.

Constructing the benchmark inputs. Nine inputs adapt eight DUCS sources. PC1 and PC2 are production-cell variants sharing one parent; they retain state-gated arrivals and make tool results UC. Pending results and reset/arrival chains finish when work/output is withheld. Adaptations shorten Railcab transfers but retain both outcomes, omit a duplicate initial-environment transfer declaration in PowerPlant and alter safety/ordering. Equivalence is unestablished (Technical Appendix O).

MTSA synthesizes fixed endpoints from FSP and prepares monitors and initialization tables before search. Interval requirements use a constant post-entry state; physical observers survive commands and transfers. The Cell_n family extends Cell to n stations with holder-dependent initialization. S3–S4 detail these paths and their preparation costs.

Exploration variants. All methods share the game and checks. Lazy queries controllable events one at a time, including all outcomes of each queried event; Eager queries all controllable events. Direct-Full constructs the reachable graph first; Full+UC also prunes controls at uncontrollable states. Update-first prioritizes update commands [18]. Technical Appendix K specifies the state-dependent query and retry orders. These orders affect exploration, not permitted execution.

Design and measures. RQ3 uses one Windows/Xeon host, a 64 GiB heap and a 1,200 s Java Virtual Machine (JVM) cap: 27×4 game cells plus 27 Legacy references. Each median requires five completed runs; unresolved trials remain capped. We report solver time including checks and total JVM time including preparation, endpoints, startup and output. Legacy uses its native timer. Mac pilots are excluded (Technical Appendices P, O.8; S4).

7.2 RQ1: Can Local Choices Change Feasibility?

We apply Section 5’s merging rules to inherited models, constructed families and a production-cell case. These populations answer different parts of the feasibility question.

Inherited inputs: no observed change. None of the 54 Base/R1 comparisons changes the decision on the nine inherited inputs. Merging reduces or preserves the state count. R2 is excluded because it names individual update events. Source inspection supports Proposition 5.3 for 12/18 transfer-only pairs. This one-way guarantee establishes safe serialization (Technical Appendix O.13); the full null result remains empirical.

Constructed mechanisms. Table 3 separates the mechanisms and their controls. Rolling and Canary account for 30 of the 33 separations by varying a shared capacity obstruction. Rolling compares individual with simultaneous transfer; its separate threshold sweep checks intermediate batch sizes. Threads contrasts finite uncontrollable activity with persistent arrivals. Complete definitions and controls are in Technical Appendix O.11.

Table 3. Granularity mechanisms, parameter sweeps and controls. Each pair compares fine and merged contracts of one model: Policy merges boundaries, other rows merge transfers. Counts are comparisons, not independent applications. The main total excludes Cell.

Family / control	Mechanism and parameters	Pairs	WIN / LOSS	LOSS / LOSS	WIN / WIN	Inc.
<i>Constructed families and parameter sweeps</i>						
Rolling	Readiness during startup. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Canary	Failure reports and recovery. $n = 2 \dots 6, 1 \leq m < n$	15	15	0	0	0
Policy	Audit overlap; role exclusion. 2 rule pairs	1	1	0	0	0
DB-Rolling	Role-gated replacement. $n = 3, m = 2, 3$	2	1	1	0	0
Rolling+Audit	Readiness and audit overlap. $n = 2, m = 1$; 1 audit pair	1	1	0	0	0
<i>Quiescence control</i>						
Threads (control)	Backpressure or persistent arrival. $n = 2 \dots 6$; queue 1,2	20	0	10	10	0
<i>Source-derived attempt</i>						
PC2-Rolling	Availability during calibration. 2 arms; floor 1	1	0	0	0	1
Main total		55	33	11	10	1
Cell _n (reference)	Empty-station transfer guards. $n = 2 \dots 10$	9	9	0	0	0

Outcomes read fine / merged; Inc. is unresolved. PC2’s merged solver remains TO, not LOSS. Rolling and Canary share a capacity obstruction: their sweeps supply 30 of the 33 WIN/LOSS pairs. n counts replicas/workers/stations. Canary bounds reported failures by $n - m$; otherwise m is an availability floor. Threads varies queue capacity 1,2 under both arrival regimes. Rolling+Audit’s boundary merge remains WIN and adds no pair here.

Answer. Separate commands can preserve capacity, necessary intermediate stages and compatible requirement lifetimes. Merging can also help by bypassing a harmful mixed state (Section 5). The constructed examples establish both possibilities; the inherited comparisons show no change.

7.2.1 A Source-Derived Calibration Case. PC2-Rolling adds calibration while requiring another arm operational. Requests are controllable, completions UC, and transfers enter calibration while preserving initializer observers. Erasing calibration recovers the original operational process, without whole-problem equivalence. Endpoints retain baseline requirements plus availability; the interval additionally treats transfers as calibration entry. The 25 boundaries and two transfers have no supplied order (Technical Appendix J).

The saved policy calibrates one arm, waits for completion, and then calibrates the other. It covers all 126 entries and keeps at least 1 arm outside calibration. Every outcome decreases a rank bounded by 37, and Link checks the connected controller. The thirteen source invariants hold from their individual starts through the remaining start-only suffixes. This checks the reached tuples; GSM below also exercises ordinary actions after activation.

Including the past changes that conclusion: 87 of 126 entries fail the history-wide interpretation, and all admit an earlier violating cycle that restores the observed values. Coverage remains limited to reached activations; Technical Appendix J gives the full partition and the unchecked initializer population.

Why the fixed merge loses. Every old entry admits both raw transfers. Before joint transfer, old plants lack calibration actions and all other steps preserve readiness. The joint transfer therefore puts both arms in calibration, clearing both interval readiness bits and reaching error. This proves impossibility, while the merged solver and fine Direct-Full both time out at 7200 s/200 GiB.

Restricted-transfer controls, including both-LOSS and incomplete cases, are in Technical Appendix O and S1–S4.

7.3 RQ2: What Do the Checks Establish?

Answer. The checks support the finite-game results and selected monitor interpretations. Table 4 identifies their populations and connects them to the guarantee layers in Table 2.

Table 4. RQ2 validation populations, keyed to the scopes in Table 2.

Scope and check	Population	Established result and boundary
Game: independent games	46 fixtures, two solvers: 92 jobs	36 WIN, 48 LOSS, 8 invalid; all 84 eligible jobs pass certificate/exhaustive checks.
Invariant from start: New-requirement lookup tables	46 source positions, 138 occurrences; 900 non-error cells	Exact start-scoped languages under reference observations (Proposition 4.1); actual activation conformance unchecked.
From update entry: Interval initialization	135 occurrences: 46 R1, 89 R2	Exact entry-scoped initialization from source guards and stage fluents; execution is unchecked.
Game: application policies	25 Lazy WIN contracts; 125 runs	Certificates check the shared Java game. Runs save summaries; complete PC2-Rolling/Railcab exports are separate.

Checking the game. Python reconstructs explicit successors for the 92-job suite. All winning jobs pass Link, which checks the connected controllers. Deliberately invalid cases omit events, deadlock or suppress UC behavior. This suite shares frontend and endpoint/goal code. The Six additional monitor-free fixtures independently reconstruct FSP endpoints, games and goals (S4.2).

Checking a requirement from its start. Initial truth and step checks establish by induction that a monitor follows the reference observation rules. Safe lookup reachability then supplies Proposition 4.1’s premise: from the selected state, the monitor accepts exactly the continuations satisfying the invariant from activation. Table 4 separates source positions, occurrences and lookup cells.

Checking duties inherited from a history. For all 138 new-requirement occurrences, residual inclusion preserves obligations from the admitted safe prior histories. Interval monitors instead begin their reference at update entry. These checks concern different declared lifetimes (Technical Appendix D).

GSM after activation. For GSM’s change from $\square\text{-decode.3}$ to $\square\text{-decode.2}$, a source-derived policy covers every entry. It starts the new requirement with observer/monitor zero, permits *decode.3*

and output before transfer, and rejects *decode.2*. It satisfies R1’s gap restriction and reaches a matching endpoint within twelve events, establishing the invariant from activation through ordinary behavior. Technical Appendix E gives the path, rank proof and separate 45-pair entry-initialization check.

Scope of the monitor checks. Connecting these reference checks to every application policy requires a further step. Actual activation decoding, post-start lookup keys, observations and fallback behavior remain unchecked. The adapter preserves physical observers across commands while reference action predicates can reset; conservativeness is unproved. Safe-history nonemptiness is also unverified except for one PC2 tuple. Thus the results support the generated games and selected reference interpretations, not source semantics for every winning policy or arbitrary FSP compilation.

What application WIN and LOSS mean. Railcab’s Base policy covers 22 entries with 1–4 ordinary gap events; R1’s has none and retains seven interval guards. Both pass certificate/Link. WIN alone supplies no gap duties; explicit interval requirements let the same synthesis check a gap-protection policy (Technical Appendix O.9).

A Railcab transfer illustrates why the policy must branch: its two outcomes require different remaining paths to different new-controller states. Both paths complete and decrease rank; S4.4.3 gives the full fragment.

Exhaustive closure shows Industry/R1 loses at 42/131 entries under every order: validating either of two components violates new requirements; another component may reject forever, or deadlock quiescently if safe validation is withheld. Native DUCS wins its different subsequent-control problem (Technical Appendix O; S4).

S4 and Technical Appendix O.8 report the certificates’ stable progress bounds and state counts separately from the linked controllers.

7.4 RQ3: Does Lazy Reduce Exploration?

Answer. Lazy decides 26 contracts and Eager 17 under the same uncontrollable-event pruning. Against Direct-Full, all completed pairs agree on the decision and Lazy uses fewer states and queries (Table 5). This comparison also changes pruning, so it measures both construction choices. Update-first is faster in 8 of 24 completed pairs but adds timeouts. The table reports both solver and total JVM ratios; preparation can substantially reduce the apparent gain.

Lazy proves Industry/R1 losing and times out on PC2/R2. PC2/R2 also times out with 3,600 s and 64 GiB, and in the earlier series with 7,200 s and 200 GiB; the limiting resource is unknown (S4).

Legacy returns 9 WIN, 13 out-of-memory (OOM) failures and 5 unmeasured snapshot-cap cases. Because it also synthesizes subsequent control, these are unmatched references (Technical Appendix O).

Figure 2 shows the absolute costs of all 14 completed pairs; per-contract results are in Technical Appendices M and O.

Scope of the cost comparison. The different completed populations do not isolate an interval effect. Capped totals stay outside paired ratios. Full+UC trials mix builds, hosts and repetitions, and an unpruned on-the-fly variant is unimplemented, so pruning is not isolated either. S4 and Technical Appendix O.10 retain preparation costs and auxiliary trials.

Scaling limits. All methods finish the independent family, but all encounter Travel timeouts and out-of-memory failures, including expanded-budget timeouts. These separate trials never enter medians; Technical Appendix O.10 and S4 retain all populations and outcomes.

Table 5. All 27 contracts per method. TO denotes one capped trial, not a timing median. Ratios summarize heterogeneous completed pairs; values above one favor Lazy.

Method	WIN/LOSS/TO	Faster / pairs	Solver ratio	Total JVM ratio
Lazy	25/1/1	—	—	—
Eager	17/0/10	0 / 17	2.04 [1.18–4,575.05]	1.60 [1.00–144.85]
Update-first	23/1/3	8 / 24	1.07 [0.19–39.76]	1.00 [0.32–6.10]
Direct-Full	14/0/13 [†]	0 / 14	9.14 [1.58–2,123.81]	2.27 [1.00–130.66]

TO: timeout; no OOM/unmeasured cases. [†] Workflow/R2 retains its interruption and later same-setting timeout (S4). Comparator/Lazy ratios are median [min–max] across completed pairs of five-run medians. Faster / pairs counts lower comparator solver medians among all completed pairs. JVM includes frontend, startup, endpoints and output; solver includes checking. Unresolved costs have no ratio.

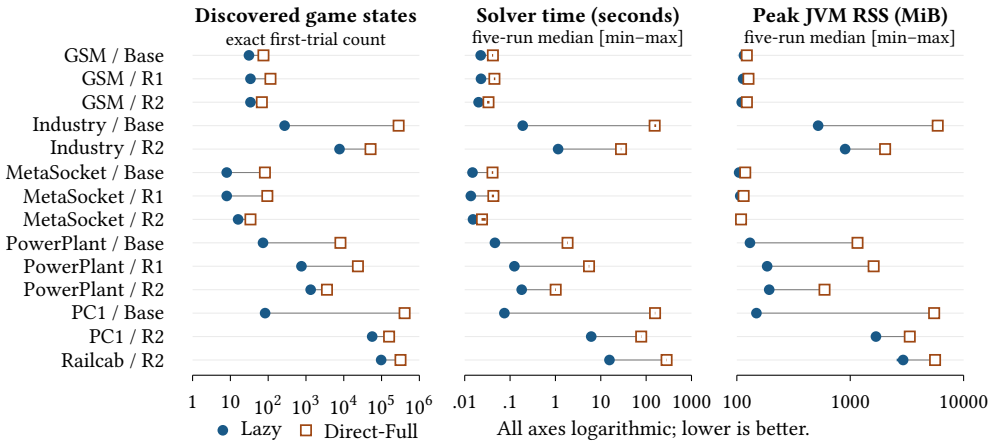


Fig. 2. All 14 completed WIN/WIN pairs. Solver time includes checking but excludes frontend, endpoint preparation and JVM startup. Resident set size (RSS) is sampled over the whole JVM run; it is neither solver-only nor heap usage. The other 13 Direct-Full contracts time out (Figure A4) and are omitted. This compares whole procedures.

8 Related Work

Generating update environments. Update-environment generation from supplied global state-preservation transitions precedes FG-DUCS [44, Sec. V]. DUCS adds reconfigure edges through possibly partial R , without a UC-quiescence guard [34, Sec. 4.2, Defs. 3.7, 5.1]. FG-DUCS requires global UC quiescence at commands and handover.

On-demand synthesis. Directed controller synthesis (DCS) already combines lazy products and controls, UC priority and backward propagation [9, Sec. IV]. Its 2016 formulation requires safety and eventual goal occurrence on every trace; the later nonblocking formulation (OTF-DCS, 2023) requires a marked extension of each finite controlled trace [10, Def. 4]. FG-DUCS specializes these patterns to its contract, with full outcome buckets and all-entry certificates. Other approaches structure specifications compositionally or modularly [31, 43].

Supervisor switching. Supervisor switching addresses safety and deadlock freedom [30, Sec. 3.3], switching speed [20, Sec. III], startup bounds [35] and mode invariants [36]. FG-DUCS addresses staged handover to a supplied controller.

Requirements across adaptation. Obligations across change occur in temporal adaptation, scenario updatability and Live Synthesis [13, 14, 27, 45]; FG-DUCS uses scoped requirements and residual reasoning [12, 37]. Thuijsman and Reniers [40, Secs. 4–7] support individual/simultaneous re-configuration, feature-dependent requirements and state retention/reset. Their guarantee requires a marked continuation from every reachable state; FG-DUCS requires all maximal update runs to finish at a supplied controller.

Safe adaptation paths. Safe adaptation graphs support minimum-cost paths [46, Secs. 3–4]. Re-configuration sequences preserve lifecycle and architectural constraints, reaching the target without failures or a consistent architecture after recovery [5, Secs. II–VI]. FG-DUCS instead checks finite handover to a fixed controller under declared requirement lifetimes; it does not optimize cost or repair failures.

Operational update mechanisms. Local, live, workflow and network updating address migration and transition safety [15, 16, 26, 28, 38]. Transaction version consistency [24] is a separate guarantee that finite handover does not imply.

9 Threats to Validity

Execution and monitor premises are unvalidated on a running platform (Section 4). Persistent uncontrollable activity can prevent completion, and the contract excludes failure repair. Constructed cases assume finite startup, reporting, recovery and election; selection may favor separations. Their deployed prevalence and the feasibility effects of model adaptation are unestablished.

Shared authorship, AI and code risk correlated faults; independent parsing covers restricted FSP. Direct-Full is in-house, and external solvers and randomized orders are untested. Extended trials lack medians. Sampling process memory can miss peaks and does not identify heap use or bottlenecks; S4 retains interruptions.

The null comparisons, losses, timeouts and faster or smaller alternatives reported in Section 7 bound the gains. Timings use one machine, and fixtures/repeats are not independent applications. Inherited inputs lack precedence and only Railcab transfers branch; exponential separation assumes the default order.

10 Conclusion

Verified endpoints leave the handover between them unresolved. FG-DUCS constructs that handover from local component transfers, requirement lifetimes and initial obligations. A winning policy safely completes from every old entry under every modeled outcome and preserves the selected new controller’s continuation. We identify obstructions to merged commands and a sufficient condition for serializing merged transfers.

The implementation decides 26 of 27 adapted games within 1,200 s. Constructed examples separate granularities in both directions; 54 inherited-model comparisons show no change. These results support finite maintenance planning that permits uncontrollable quiescence. Actual activation conformance remains unverified, and the contract gives neither continuous-service nor elapsed-time bounds.

Data Availability

We created and publicly released the anonymous FG-DUCS replication package for this paper [4]. The retained evidence snapshot is available at <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-invariants-20261002>. The paper-matched release cited in [4], fgducs-c107-20261003, contains this revised main paper and paper/supplementary_material.pdf: Part I is Technical Appendix A–P; Part II is Supplement S1–S5. Its reading guide, bookmarks and continuous page numbers locate the proofs, full models and experimental records cited throughout the main text. The repository README links directly to both parts and maps paper sections to evidence. The package also provides source code, all 27 application contracts, scripts and saved results; saved-evidence checks require no new performance experiments. Dependencies come from official upstream sources.

References

- [1] Bowen Alpern and Fred B. Schneider. 1985. Defining Liveness. *Inform. Process. Lett.* 21, 4 (1985), 181–185. doi:10.1016/0020-0190(85)90056-0
- [2] Gal Amram, Shahar Maoz, Itai Segall, and Matan Yossef. 2022. Dynamic Update for Synthesized GR(1) Controllers. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, USA, 786–797. doi:10.1145/3510003.3510054
- [3] Jesper Andersson, Luciano Baresi, Nelly Bencomo, Rogério de Lemos, Alessandra Gorla, Paola Inverardi, and Thomas Vogel. 2013. Software Engineering Processes for Self-Adaptive Systems. In *Software Engineering for Self-Adaptive Systems II*, Rogério de Lemos, Holger Giese, Hausi A. Müller, and Mary Shaw (Eds.). Lecture Notes in Computer Science, Vol. 7475. Springer, Berlin, Germany, 51–75. doi:10.1007/978-3-642-35813-5_3
- [4] Anonymous FG-DUCS Authors. 2026. FG-DUCS Evaluation Artifact. Public anonymous replication package prepared for this paper, including Technical Appendix A–P, Supplement S1–S5, source code and saved evidence; tag fgducs-c107-20261003. <https://github.com/research-artifact-archive/research-artifact/tree/fgducs-c107-20261003>
- [5] Fabienne Boyer, Olivier Gruber, and Damien Pous. 2013. Robust Reconfigurations of Component Assemblies. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 13–22. doi:10.1109/ICSE.2013.6606547
- [6] Franck Cassez, Alexandre David, Emmanuel Fleury, Kim G. Larsen, and Didier Lime. 2005. Efficient On-the-Fly Algorithms for the Analysis of Timed Games. In *CONCUR 2005 – Concurrency Theory (Lecture Notes in Computer Science, Vol. 3653)*. 66–80. doi:10.1007/11539452_9
- [7] Betty H. C. Cheng, Rogério de Lemos, Holger Giese, Paola Inverardi, Jeff Magee, Jesper Andersson, Basil Becker, Nelly Bencomo, Yuriy Brun, Bojan Cukic, Giovanna Di Marzo Serugendo, Schahram Dustdar, Anthony Finkelstein, Cristina Gacek, Kurt Geihs, Vincenzo Grassi, Gabor Karsai, Holger M. Kienle, Jeff Kramer, Marin Litoiu, Sam Malek, Raffaella Mirandola, Hausi A. Müller, Sooyong Park, Mary Shaw, Matthias Tichy, Massimo Tivoli, Danny Weyns, and Jon Whittle. 2009. Software Engineering for Self-Adaptive Systems: A Research Roadmap. In *Software Engineering for Self-Adaptive Systems*. Lecture Notes in Computer Science, Vol. 5525. Springer, Berlin, Germany, 1–26. doi:10.1007/978-3-642-02161-9_1
- [8] Alessandro Cimatti, Marco Roveri, and Paolo Traverso. 1998. Strong Planning in Non-Deterministic Domains via Model Checking. In *Proceedings of the Fourth International Conference on Artificial Intelligence Planning Systems (AIPS '98)*. AAAI Press, Menlo Park, CA, USA, 36–43. <https://cdn.aaai.org/AIPS/1998/AIPS98-005.pdf>
- [9] D. Ciolek, V. Braberman, N. D'Ippolito, and S. Uchitel. 2016. *Directed controller synthesis of discrete event systems*. Technical Report 1605.09772. arXiv.
- [10] Daniel Ciolek, Matias Duran, Florencia Zanollo, Nicolás Pazos, Julián Braier, Víctor Braberman, Nicolás D'Ippolito, and Sebastián Uchitel. 2023. On-the-Fly Informed Search of Non-Blocking Directed Controllers. *Automatica* 147 (2023), 110731. doi:10.1016/j.automatica.2022.110731
- [11] Nicolas D'Ippolito, Dario Fischbein, Marsha Chechik, and Sebastian Uchitel. 2008. MTSA: The Modal Transition System Analyser. In *2008 23rd IEEE/ACM International Conference on Automated Software Engineering*. IEEE, Piscataway, NJ, USA, 475–476. doi:10.1109/ASE.2008.78
- [12] Matthew B. Dwyer, George S. Avrunin, and James C. Corbett. 1999. Patterns in Property Specifications for Finite-State Verification. In *Proceedings of the 21st International Conference on Software Engineering (ICSE '99)*. ACM, New York, NY, USA, 411–420. doi:10.1145/302405.302672
- [13] Bernd Finkbeiner, Felix Klein, and Niklas Metzger. 2022. Live Synthesis. *Innovations in Systems and Software Engineering* 18, 3 (2022), 443–454. doi:10.1007/s11334-022-00447-5

- [14] Carlo Ghezzi, Joel Greenyer, and Valerio Panzica La Manna. 2012. Synthesizing Dynamically Updating Controllers from Changes in Scenario-Based Specifications. In *Proceedings of the 7th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '12)*. IEEE, Piscataway, NJ, USA, 145–154. doi:10.1109/SEAMS.2012.6224401
- [15] Cristiano Giuffrida, Anton Kuijsten, and Andrew S. Tanenbaum. 2013. Safe and Automatic Live Update for Operating Systems. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '13)*. ACM, New York, NY, USA, 279–292. doi:10.1145/2451116.2451147
- [16] Christopher M. Hayden, Karla Saur, Edward K. Smith, Michael Hicks, and Jeffrey S. Foster. 2014. Kitsune: Efficient, General-Purpose Dynamic Software Updating for C. *ACM Transactions on Programming Languages and Systems* 36, 4, Article 13 (Oct. 2014), 38 pages. doi:10.1145/2629460
- [17] Maurice P. Herlihy and Jeannette M. Wing. 1990. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems* 12, 3 (1990), 463–492. doi:10.1145/78969.78972
- [18] Takanori Hirano, Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2024. Dynamic Update Controller Synthesis using on-the-fly Exploration. *IEICE Technical Report* 123, 443 (March 2024), 76–78. KBSE2023-78; in Japanese; non-peer-reviewed technical report. <https://ken.ieice.org/ken/paper/20240314Vccd/eng/>
- [19] Martin Hofmann, Christian Neukirchen, and Harald Rueß. 2016. Certification for μ -Calculus with Winning Strategies. In *Model Checking Software (Lecture Notes in Computer Science, Vol. 9641)*. Springer, Cham, Switzerland, 111–128. doi:10.1007/978-3-319-32582-8_8
- [20] Ting Jiao, Yongmei Gan, and Guochun Xiao. 2013. On the Reconfiguration of Flexible Manufacturing Workcells. In *2013 IEEE International Conference of IEEE Region 10 (TENCON 2013)*. IEEE, 1–4. doi:10.1109/TENCON.2013.6718470
- [21] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Transactions on Software Engineering* 16, 11 (1990), 1293–1306. doi:10.1109/32.60317
- [22] Orna Kupferman and Moshe Y. Vardi. 2001. Model Checking of Safety Properties. *Formal Methods in System Design* 19, 3 (2001), 291–314. doi:10.1023/A:1011254632723
- [23] Claus Lewerentz and Thomas Lindner. 1995. Case Study “Production Cell”: A Comparative Study in Formal Specification and Verification. In *KORSO: Methods, Languages, and Tools for the Construction of Correct Software*, Manfred Broy and Stefan Jähnichen (Eds.). Lecture Notes in Computer Science, Vol. 1009. Springer, Berlin, Germany, 388–416. doi:10.1007/BFb0015473
- [24] Xiaoxing Ma, Luciano Baresi, Carlo Ghezzi, Valerio Panzica La Manna, and Jian Lu. 2011. Version-consistent Dynamic Reconfiguration of Component-based Distributed Systems. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ACM, 245–255. doi:10.1145/2025113.2025148
- [25] Jeff Magee and Jeff Kramer. 2006. *Concurrency: State Models and Java Programs* (2 ed.). John Wiley & Sons, Chichester, UK. <https://www.doc.ic.ac.uk/~jnm/book/>
- [26] Valerio Panzica La Manna. 2012. Local Dynamic Update for Component-Based Distributed Systems. In *Proceedings of the 15th ACM SIGSOFT Symposium on Component Based Software Engineering (CBSE '12)*. ACM, New York, NY, USA, 167–176. doi:10.1145/2304736.2304764
- [27] Valerio Panzica La Manna, Joel Greenyer, Carlo Ghezzi, and Christian Brenner. 2013. Formalizing Correctness Criteria of Dynamic Updates Derived from Specification Changes. In *Proceedings of the 8th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '13)*. IEEE, Piscataway, NJ, USA, 63–72. doi:10.1109/SEAMS.2013.6595493
- [28] Jedidiah McClurg, Hossein Hojjat, Pavol Černý, and Nate Foster. 2015. Efficient Synthesis of Network Updates. In *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*. ACM, New York, NY, USA, 196–207. doi:10.1145/2737924.2737980
- [29] Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. 2011. Certifying Algorithms. *Computer Science Review* 5, 2 (2011), 119–161. doi:10.1016/j.cosrev.2010.09.009
- [30] Rômulo Meira-Góes and Stéphane Lafortune. 2020. Moving Target Defense based on Switched Supervisory Control: A New Technique for Mitigating Sensor Deception Attacks. *IFAC-PapersOnLine* 53, 4 (2020), 317–323. doi:10.1016/j.ifacol.2021.04.031
- [31] Sahar Mohajerani, Robi Malik, and Martin Fabian. 2014. A Framework for Compositional Synthesis of Modular Nonblocking Supervisors. *IEEE Trans. Automat. Control* 59, 1 (2014), 150–162. doi:10.1109/TAC.2013.2283109
- [32] MongoDB. [n. d.]. Upgrade to MongoDB Enterprise (Replica Set). MongoDB Database Manual, version 7.0. Accessed 2026-09-29. <https://www.mongodb.com/docs/v7.0/tutorial/upgrade-to-enterprise-replica-set/>
- [33] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2016. Assured and Correct Dynamic Update of Controllers. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS '16)*. ACM, New York, NY, USA, 96–107. doi:10.1145/2897053.2897056

- [34] Leandro Nahabedian, Víctor Braberman, Nicolás D’Ippolito, Shinichi Honiden, Jeff Kramer, Kenji Tei, and Sebastián Uchitel. 2020. Dynamic update of discrete event controllers. *IEEE Transactions on Software Engineering* 46, 11 (2020), 1220–1240. doi:10.1109/TSE.2018.2876843
- [35] Anas Nooruldeen and Klaus Werner Schmidt. 2015. State Attraction Under Language Specification for the Reconfiguration of Discrete Event Systems. *IEEE Trans. Automat. Control* 60, 6 (2015), 1630–1634. doi:10.1109/TAC.2014.2358811
- [36] Spyros Reveliotis and Zhennan Fei. 2017. Invariant-Based Supervisory Control of Switched Discrete Event Systems. *IEEE Trans. Automat. Control* 62, 2 (2017), 921–927. doi:10.1109/TAC.2016.2560536
- [37] Grigore Roşu and Klaus Havelund. 2005. Rewriting-Based Techniques for Runtime Verification. *Automated Software Engineering* 12, 2 (2005), 151–197. doi:10.1007/s10515-005-6205-y
- [38] Daniel Sokolowski, Pascal Weisenburger, and Guido Salvaneschi. 2022. Change Is the Only Constant: Dynamic Updates for Workflows. In *Proceedings of the 44th International Conference on Software Engineering (ICSE ’22)*. ACM, New York, NY, USA, 350–362. doi:10.1145/3510003.3510065
- [39] The Kubernetes Authors. [n. d.]. Deployments. Kubernetes Documentation. Accessed 2026-09-29. <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [40] Sander B. Thuijsman and Michel A. Reniers. 2024. Supervisory Control for Dynamic Feature Configuration in Product Lines. *ACM Transactions on Embedded Computing Systems* 23, 5, Article 71 (2024), 25 pages. doi:10.1145/3579644
- [41] Stavros Tripakis and Karine Altisen. 1999. On-the-Fly Controller Synthesis for Discrete and Dense-Time Systems. In *Formal Methods (Lecture Notes in Computer Science, Vol. 1708)*. Springer, Berlin, Germany, 233–252. doi:10.1007/3-540-48119-2_15
- [42] Yves Vandewoude, Peter Ebraert, Yolande Berbers, and Theo D’Hondt. 2007. Tranquility: A Low Disruptive Alternative to Quiescence for Ensuring Safe Dynamic Updates. *IEEE Transactions on Software Engineering* 33, 12 (2007), 856–868. doi:10.1109/TSE.2007.70733
- [43] W. Murray Wonham and Peter J. Ramadge. 1988. Modular Supervisory Control of Discrete-Event Systems. *Mathematics of Control, Signals, and Systems* 1, 1 (1988), 13–30. doi:10.1007/BF02551233
- [44] Takuto Yamauchi, Kenji Tei, and Shinichi Honiden. 2020. Method for Low-Cost Environment Partitioning Modeling in Dynamic Update. In *Proceedings of the 2020 IEEE Third International Conference on Artificial Intelligence and Knowledge Engineering (AIKE ’20)*. IEEE, Piscataway, NJ, USA, 183–187. doi:10.1109/AIKE48582.2020.00036
- [45] Ji Zhang and Betty H. C. Cheng. 2006. Using Temporal Logic to Specify Adaptive Program Semantics. *Journal of Systems and Software* 79, 10 (2006), 1361–1369. doi:10.1016/j.jss.2006.02.062
- [46] Ji Zhang, Betty H. C. Cheng, Zhenxiao Yang, and Philip K. McKinley. 2005. Enabling Safe Dynamic Component-Based Software Adaptation. In *Architecting Dependable Systems III*, Rogério de Lemos, Cristina Gacek, and Alexander Romanovsky (Eds.). Lecture Notes in Computer Science, Vol. 3549. Springer, Berlin, Heidelberg, 194–211. doi:10.1007/11556169_9